Smart Vaccine Delivery System

IoT-Based Cold-Chain Monitoring & Safety Analytics

Course EC-308: Internet of Things

Presented to:
Dr. [Professor Name]

Presented By:
Anirudh Sharma
Roll No: 22DCS002

BTech DD 8th Semester, 2025–26

Department of Computer Science and Engineering
National Institute of Technology Hamirpur

Table of Contents

Introduction & Problem Statement

Background

- Vaccines require strict cold-chain maintenance (2 ℃–8 ℃) during transport
- WHO reports up to **50% vaccine wastage** due to cold-chain failures globally
- Manual monitoring is unreliable and lacks real-time alerts
- Physical damage (drops, vibration) and gas leakage go undetected

Problem Statement

- How do we *continuously* monitor temperature, humidity, mechanical stress, and contamination during vaccine transport?
- How do we *automatically* raise alerts and log violations to the cloud in real time?

Cold Chain Risk Zones

Temperature: 2 ℃ – 8 ℃ (safe range)

Drop & Shock: mechanical impact risk

Gas & Odor: leakage or spoilage

Humidity: condensation and failure

Tamper: box opened in transit

Each factor = potential vaccine loss

Limitations

Hardware / Simulation Constraints

- **MPU6050 not in Wokwi** — simulated via potentiometers; real I2C dynamics not fully represented
- **MQ-135 warm-up** — requires 24–48 hrs in real deployment; simulation is instantaneous
- **ESP32 ADC non-linearity** — GPIO 34/35 accuracy is $\pm 6\%$ without calibration
- **ThingSpeak free-tier** — 15 s minimum upload interval; unsuitable for high-frequency events

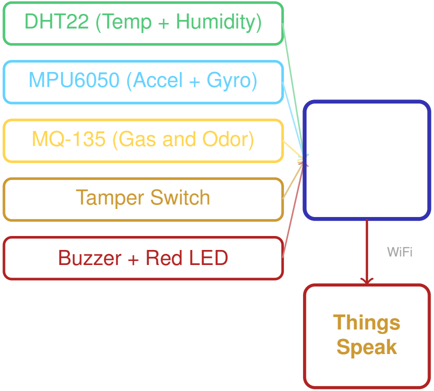
System-Level Limitations

- **No GPS** — violation location unknown; route safety analysis not possible
- **No power management** — continuous WiFi drains battery; external power needed for long trips
- **No encryption** — API key hardcoded; vulnerable if network is intercepted
- **Single point of failure** — one ESP32 failure stops all monitoring; no redundancy
- **WiFi only** — no offline buffering; data lost on connectivity drop

Proposed Solution

System Architecture

- **ESP32** MCU with built-in WiFi for edge processing & cloud upload
- **DHT22** — Temperature + Humidity (cold-chain & condensation)
- **MPU6050** — Accelerometer + Gyroscope (shock, drop, rash driving)
- **MQ-135** — Gas sensor (leakage / spoilage detection)
- **Tamper switch** — Physical security of vaccine box
- **Buzzer + LED** — Immediate on-device alerts
- **ThingSpeak** — Cloud logging with 8-field data channel
- **Web Dashboard** — Real-time gauges, charts, event simulation



Tools & Sensors

Hardware Components

Component	Purpose
ESP32 DevKit v4	Main MCU + WiFi
DHT22	Temp & Humidity sensing
MPU6050	Acceleration + Gyroscope
MQ-135	Gas / Odor detection
Push Button	Tamper detection
Passive Buzzer	Audible alarm
Red LED	Visual alert indicator
Resistor 4.7kΩ	DHT22 pull-up
Resistor 220Ω	LED current limiter

Software & Simulation Tools

- **Wokwi** — Browser-based ESP32 simulation
- **Arduino C++** — Firmware development
- **DHTesp Library** — DHT22 driver
- **WiFi.h / HTTPClient.h** — Cloud comms
- **ThingSpeak** — IoT cloud data platform
- **HTML/CSS/JS + Chart.js** — Dashboard

Simulation Note

- MPU6050 & MQ-135 simulated via **potentiometers** in Wokwi (no hardware needed)
- Full I2C / analog code available for physical deployment

Tools & Sensors — Resistors & Pin Mapping

Resistors Used

Ref.	Value	Purpose
R1	4700 Ω	DHT22 pull-up to 3.3 V
R2	220 Ω	LED current limiter

Why These Values?

- **4.7kΩ**: Standard pull-up for DHT 1-wire protocol; keeps idle line stable at 3.3 V logic
- **220Ω**: Limits LED current to $\approx$ 15 mA at 3.3 V, within safe ESP32 GPIO limit ($\leq$ 20 mA)

GPIO Pin Mapping (ESP32)

GPIO	Component	Mode	Reason
4	DHT22 Data	Digital In	1-Wire protocol
35	Accel-X (pot)	Analog In	ADC1, input-only
32	Accel-Y (pot)	Analog In	ADC1, general
34	Gas MQ-135	Analog In	ADC1, input-only
15	Tamper Button	Digital In	INPUT_PULLUP
13	Buzzer	Digital Out	PWM capable
2	Red LED	Digital Out	On-board LED

Why ADC1 (GPIO 34, 35, 32)?

- ADC2 pins conflict with WiFi when active
- GPIO 34, 35 are input-only — no accidental output

Tools & Sensors — Why These Components?

- **ESP32** — Built-in WiFi + Bluetooth, dual-core 240 MHz, multiple ADC channels, 3.3 V I/O. Essential for cloud uploads without separate shield.
- **DHT22** — Higher accuracy ($\pm$0.5 ℃, $\pm$2–5% RH) vs DHT11; covers vaccine safe range (2–8 ℃). Single-wire digital interface.
- **MPU6050** — 6-axis IMU (3-axis accel + 3-axis gyro) over I2C. Detects drops, vibration, rash driving. Low cost, well-supported.
- **MQ-135** — Broad-spectrum gas sensor: NH_3, CO_2, benzene, smoke. Detects spoilage or chemical leakage from vaccine containers.
- **Tamper Button** — Simple low-cost physical security. INPUT_PULLUP mode: no external resistor needed; reliable detection.
- **ThingSpeak** — Free REST-API cloud IoT platform; 8 fields per channel; built-in MATLAB visualisations. 15 s upload interval suits vaccine monitoring.
- **Wokwi** — Only browser-based simulator with native ESP32 + DHT22 support. No hardware required for full demonstration.
- **Chart.js Dashboard** — Lightweight, no-framework real-time visualisation; animated gauges, chart history, event simulation in plain HTML/JS.

Simulation — Wokwi Circuit

[Simulation Screenshot]

ESP32 + DHT22 + Potentiometers + Button + Buzzer + LED

Components in Wokwi

- ESP32 DevKit C v4 — Main board
- DHT22 with 4.7kΩ pull-up to 3.3 V
- **Pot 1** (GPIO 35) — Simulates Accel-X
- **Pot 2** (GPIO 32) — Simulates Accel-Y
- **Pot 3** (GPIO 34) — Simulates MQ-135 Gas
- Push Button (GPIO 15) — Tamper detection
- Passive Buzzer (GPIO 13)
- Red LED + 220Ω (GPIO 2)

Simulating Events

- Pot 1/2 $\rightarrow$ right = shock / drop
- Pot 3 $\rightarrow$ right = gas leak
- Click DHT22 $\rightarrow$ change temp/humidity
- Red button = tamper alert

Simulation — Serial Monitor Output

[Simulation Screenshot]

Wokwi Serial Monitor — ASCII dashboard output

Serial Output Fields

- Temperature (℃) with range check
- Humidity (%RH) with HIGH flag
- Acceleration magnitude with SHOCK flag
- Gyro speed with FAST flag
- Gas level with LEAK flag
- Tamper: SEALED / OPEN
- Damage Score (0–10)
- Status Code (0, 1, 2. . . 9)

JSON Data Output

- Machine-readable per 2 s cycle
- Format:
  `DATA:{temp, hum, gas, accel,...}`
- Enables direct dashboard integration

Cloud Integration — ThingSpeak

[Simulation Screenshot]

ThingSpeak channel — 8-field live graphs

Channel Data Fields

Field	Data	Unit
1	Temperature	℃
2	Humidity	%RH
3	Status Code	0–9
4	Acceleration Magnitude	raw
5	Gas Level	analog
6	Damage Score	0–10
7	Gyro Speed	raw
8	Tamper State	0/1

Upload Protocol

- HTTP GET to `api.thingspeak.com/update`
- Upload every **15 s** (free-tier minimum)
- WiFi: `Wokwi-GUEST` (built-in simulation)
- Offline mode if WiFi disconnects

Code Walkthrough — Sensor Reading & Damage Score

DHT22 + MPU6050 Simulation

```
// DHT22
TempAndHumidity d = dht.getTempAndHumidity();
temperature = d.temperature;
humidity    = d.humidity;

// MPU6050 via potentiometers
int rawX = analogRead(ACCEL_X_PIN); // 0-4095
int rawY = analogRead(ACCEL_Y_PIN);
float ax = map(rawX, 0,4095,-32768,32767);
float ay = map(rawY, 0,4095,-32768,32767);
float az = 16384; // ~1g baseline
accelMag = sqrt(ax*ax + ay*ay + az*az);

// Tamper
tampered = (digitalRead(TAMPER_PIN)==LOW);
```

Damage Score Algorithm

```
damageScore = 0; status_code = 1;

if (temp>8 || temp<2)  damageScore += 3;
if (tampered)          damageScore += 2;
if (accelMag > 20000)  damageScore += 2;
if (gyroSpeed > 30000) damageScore += 1;
if (gasValue > 400)    damageScore += 3;
if (hum > 80)          damageScore += 1;

// CRITICAL override
if (damageScore >= 5)
    status_code = 9; // CRITICAL
```

Code Walkthrough — Cloud Upload & Status Codes

ThingSpeak HTTP Upload

```
String url = server +
  "?api_key=" + apiKey +
  "&field1=" + String(temperature) +
  "&field2=" + String(humidity)    +
  "&field3=" + String(status_code) +
  "&field4=" + String(accelMag)    +
  "&field5=" + String(gasValue)    +
  "&field6=" + String(damageScore);

http.begin(url);
int code = http.GET();
// Runs every 15 seconds
```

Status Code Reference

Code	Condition	Level
0	Temp out of range	Critical
1	All normal	OK
2	Tamper detected	Warning
3	Drop / Shock	Critical
4	Unsafe driving	Warning
5	Gas leakage	Critical
6	High humidity	Warning
9	**Damage** ≥ 5	**CRITICAL**

Results

[Simulation Screenshot]

Web Dashboard — gauges, chart, status grid, alert log

Key Observations

- Cold-chain failure correctly triggers Status 0 when temp $> 8\,^{\circ}\mathrm{C}$ or $< 2\,^{\circ}\mathrm{C}$
- Shock event (pot at max) $\rightarrow$ Status 3 within 2 s; buzzer activates
- Gas leakage raises Status 5 with LED flash immediately
- Multi-fault escalates to **CRITICAL (Status 9)** via damage score
- ThingSpeak receives all 8 data fields every 15 s
- Serial monitor confirms correct JSON output each read cycle

Future Scope

Short-Term Enhancements

- **GPS (NEO-6M)** — Real-time location; correlate shocks to road segments for route safety scoring
- **Telegram / WhatsApp Bot** — Instant push alerts to driver and logistics manager on any status change
- **Deep-Sleep Mode** — Reduce ESP32 power; enable 72+ hour battery operation
- **SD Card Logging** — Store data locally during WiFi outages; upload on reconnect

Advanced Research Directions

- **Blockchain Audit Trail** — Immutable temp & violation logs on Starknet/Ethereum; tamper-proof proof-of-custody for hospitals
- **ML on Edge** — TensorFlow Lite anomaly detection on ESP32; predict failures before thresholds breach
- **LPWAN (LoRa / NB-IoT)** — Long-range, low-power connectivity for rural transport corridors
- **Fleet Monitoring** — Multi-vehicle dashboard with route-level aggregate risk scoring

Conclusion

- Designed and simulated a **complete IoT pipeline** for real-time vaccine cold-chain monitoring using ESP32
- Integrated **5 sensing modalities**: temperature, humidity, motion, gas, and tamper detection into a single embedded system
- Novel **Damage Score Algorithm** provides composite safety assessment with CRITICAL-level escalation
- Cloud connectivity via **ThingSpeak** with 8-field data logging and HTTP upload every 15 s
- **Premium real-time dashboard**: animated gauges, Chart.js history, event simulation, severity-coded alert log
- *"Ensuring vaccine integrity during transport by monitoring environmental conditions, mechanical stress, and contamination risks in real time using IoT and cloud analytics"*

Top 1% IoT Project

All pillars successfully implemented

References

Simulation & Hardware

- **Wokwi Reference**
  `https://wokwi.com`
  ESP32 DevKit C v4 docs, DHT22 component, potentiometer simulation guide
- **ESP32 Reference**
  `https://docs.espressif.com`
  Technical Reference Manual, ADC1 GPIO specs, WiFi + HTTPClient library
- **MPU6050 Datasheet**
  InvenSense MPU-6000/6050 Product Spec. Rev. 3.4
- **MQ-135 Datasheet**
  Hanwei Electronics MQ-135 Gas Sensor datasheet

Libraries, Cloud & Standards

- **DHTesp Library**
  Bernd Giesecke `beegee-tokyo/DHTesp` (GitHub)
- **ThingSpeak Platform**
  `https://thingspeak.com`
  MathWorks REST API documentation
- **Chart.js Library**
  `https://chartjs.org` — v4.4.4
  Open-source charting for real-time dashboards
- **WHO Cold-Chain Standard**
  WHO Guidelines on Vaccine Cold Chain, WHO/IVB/06.10
- **DHT22 Datasheet**
  Aosong Electronics AM2302 datasheet

Thank You

Questions & Discussion

Anirudh Sharma | Roll No: 22DCS002
BTech DD 8th Semester, 2025–26
Department of Computer Science and Engineering
National Institute of Technology Hamirpur